SQL FINGERTIPS
THE COMPLETE QUICK REFERENCE

Syntax you can see - every command written out in full, with a plain-English line under it

Prepared for: jashu5446
5 October 2026

INDEX

SQL FINGERTIPS — THE COMPLETE QUICK REFERENCE 3

SECTION 1 — THE FIVE COMMAND FAMILIES 4

SECTION 2 — THE ORDER OF EXECUTION 5

SECTION 3 — SELECT — THE FULL SYNTAX 6

SECTION 4 — FILTERING — EVERY OPERATOR WRITTEN OUT 7

SECTION 5 — JOINS — EACH ONE WRITTEN OUT 8

SECTION 6 — AGGREGATES, GROUP BY, HAVING 9

SECTION 7 — SUBQUERIES 10

SECTION 8 — WINDOW FUNCTIONS 11

SECTION 9 — DDL — CREATE, ALTER, DROP 12

SECTION 10 — DML — INSERT, UPDATE, DELETE 13

SECTION 11 — TRANSACTIONS 14

SECTION 12 — THE DIFFERENCES (the direct "what's the difference" questions) 15

SECTION 13 — THE QUERIES THEY ASK FOR (visible syntax) 16

SECTION 14 — HANDY FUNCTIONS 17

SECTION 15 — POWER COMBOS 18

SQL FINGERTIPS — THE COMPLETE QUICK REFERENCE

Syntax you can SEE — every command written out in full, with a plain-English line under it. No hunting through tables.

SECTION 1 — THE FIVE COMMAND FAMILIES

```
DDL  (structure)     CREATE   ALTER   DROP   TRUNCATE
DML  (data)          INSERT   UPDATE  DELETE
DQL  (read)          SELECT
DCL  (permissions)   GRANT    REVOKE
TCL  (transactions)  COMMIT   ROLLBACK   SAVEPOINT
```

DDL changes the SHAPE of the table, DML changes the ROWS, DQL reads, DCL controls who may do what, and TCL controls transactions. Note: TRUNCATE looks like a delete but it is DDL.

SECTION 2 — THE ORDER OF EXECUTION

You WRITE a query in this order:

```
SELECT   columns
FROM     table
WHERE    row_condition
GROUP BY column
HAVING   group_condition
ORDER BY column
LIMIT    n;
```

The database RUNS it in this order:

```
1.  FROM / JOIN       build the working rows
2.  WHERE             filter individual ROWS
3.  GROUP BY          collapse rows into GROUPS
4.  HAVING            filter the GROUPS
5.  SELECT            compute the output columns
6.  DISTINCT          remove duplicate result rows
7.  ORDER BY          sort the result
8.  LIMIT / OFFSET    take the requested slice
```

What this tells you: WHERE filters ROWS and **cannot** use an aggregate (COUNT / SUM / AVG / MIN / MAX) — those go in HAVING. And a SELECT alias is not visible in WHERE, because SELECT has not run yet.

SECTION 3 — SELECT — THE FULL SYNTAX

```
SELECT [DISTINCT] column1, column2, aggregate(column) AS alias
FROM   table_name
WHERE  row_condition
GROUP BY column1
HAVING aggregate_condition
ORDER BY column1 [ASC | DESC]
LIMIT  n OFFSET m;
```

`ASC` is ascending (default), `DESC` is descending. `AS alias` renames the output column. Only the clauses you need are required — the rest are optional.

SECTION 4 — FILTERING — EVERY OPERATOR WRITTEN OUT

```
WHERE salary  = 50000                   -- equal
WHERE salary  <> 50000                  -- not equal ( != also works)
WHERE salary  >  50000                  -- greater than
WHERE salary  >= 50000                  -- greater than or equal
WHERE salary  BETWEEN 40000 AND 80000   -- inclusive on BOTH ends
WHERE dept_id IN (1, 2, 3)              -- in a list
WHERE dept_id NOT IN (1, 2)             -- not in a list
WHERE emp_name LIKE 'A%'                -- starts with A   ( % = any number of chars )
WHERE emp_name LIKE '%a'                -- ends with a
WHERE emp_name LIKE '_avi'              -- exactly one char, then avi  ( _ = one char )
WHERE dept_id IS NULL                   -- NULL test — never use  = NULL
WHERE dept_id IS NOT NULL               -- not null
```

AND binds tighter than OR, so bracket your intent:

```
WHERE (dept_id = 1 OR dept_id = 2) AND salary > 60000
```

SECTION 5 — JOINS — EACH ONE WRITTEN OUT

```
-- INNER JOIN — only rows that match on both sides
SELECT e.emp_name, d.dept_name
FROM   employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;

-- LEFT JOIN — every LEFT row, plus matches (no match -> NULL)
SELECT e.emp_name, d.dept_name
FROM   employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;

-- RIGHT JOIN — every RIGHT row, plus matches
SELECT e.emp_name, d.dept_name
FROM   employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;

-- FULL OUTER JOIN — both sides (MySQL: write it as LEFT UNION RIGHT)
SELECT e.emp_name, d.dept_name
FROM   employees e LEFT JOIN departments d ON e.dept_id = d.dept_id
UNION
SELECT e.emp_name, d.dept_name
FROM   employees e RIGHT JOIN departments d ON e.dept_id = d.dept_id;

-- SELF JOIN — the table joined to itself (employee -> manager)
SELECT w.emp_name AS employee, m.emp_name AS manager
FROM   employees w
LEFT JOIN employees m ON w.manager_id = m.emp_id;

-- ANTI-JOIN — rows with NO match (departments with no employees)
SELECT d.dept_name
FROM   departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
WHERE  e.emp_id IS NULL;
```

Inner keeps the overlap. Left keeps the left side. Right keeps the right side. Full keeps both. Self joins a table to itself. The anti-join finds what is missing.

SECTION 6 — AGGREGATES, GROUP BY, HAVING

```
SELECT   dept_id, COUNT(*) AS headcount, AVG(salary) AS avg_salary
FROM     employees
WHERE    salary > 40000            -- filter ROWS first
GROUP BY dept_id                   -- then make GROUPS
HAVING   COUNT(*) > 1              -- then filter GROUPS
ORDER BY avg_salary DESC;          -- sort last
```

The aggregate functions:

```
COUNT(*)                -- number of ROWS in the group (counts NULLs too)
COUNT(column)           -- number of NON-NULL values
COUNT(DISTINCT column)  -- number of unique NON-NULL values
SUM(column)             -- total
AVG(column)             -- average of the non-NULL values
MIN(column)             -- smallest
MAX(column)             -- largest
```

The rule: every column in SELECT must be inside an aggregate OR listed in GROUP BY. And remember what the aggregate counts — `COUNT(*)` counts the ROWS in each group; the grouped column only decides which bucket a row falls into.

SECTION 7 — SUBQUERIES

```
-- scalar (returns one value)
SELECT emp_name FROM employees
WHERE  salary > (SELECT AVG(salary) FROM employees);

-- list
SELECT emp_name FROM employees
WHERE  dept_id IN (SELECT dept_id FROM departments WHERE location = 'Mumbai');

-- ALL = greater than the max;  ANY = greater than the min
WHERE salary > ALL (SELECT salary FROM employees WHERE dept_id = 2)
WHERE salary > ANY (SELECT salary FROM employees WHERE dept_id = 2)

-- correlated (re-runs for every outer row)
SELECT e.emp_name FROM employees e
WHERE  e.salary > (SELECT AVG(salary) FROM employees WHERE dept_id = e.dept_id);

-- EXISTS (stops at the first match, NULL-safe)
SELECT d.dept_name FROM departments d
WHERE  EXISTS (SELECT 1 FROM employees e WHERE e.dept_id = d.dept_id);
```

Small inner list -> IN. Large inner table -> EXISTS.

SECTION 8 — WINDOW FUNCTIONS

```
SELECT emp_name, salary,
       ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rn,
       RANK()       OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk,
       DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dense,
       LAG(salary)  OVER (ORDER BY hire_date)  AS previous_salary,
       LEAD(salary) OVER (ORDER BY hire_date)  AS next_salary,
       SUM(salary)  OVER (ORDER BY hire_date)  AS running_total
FROM   employees;
```

```
ROW_NUMBER()  ->  1, 2, 3, 4     always unique
RANK()        ->  1, 2, 2, 4     ties share, then SKIPS
DENSE_RANK()  ->  1, 2, 2, 3     ties share, no skip
```

PARTITION BY resets the calculation per group. ORDER BY inside OVER makes it cumulative. A window function keeps EVERY row (unlike GROUP BY, which collapses them).

Top-N per group:

```
SELECT * FROM (
  SELECT emp_name, dept_id, salary,
         ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rn
  FROM   employees
) t
WHERE rn <= 2;
```

SECTION 9 — DDL — CREATE, ALTER, DROP

```
CREATE TABLE employees (
    emp_id   INT PRIMARY KEY,
    emp_name VARCHAR(50) NOT NULL,
    dept_id  INT,
    salary   DECIMAL(10,2) CHECK (salary > 0),
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);

ALTER TABLE employees ADD COLUMN email VARCHAR(100);
ALTER TABLE employees MODIFY COLUMN salary DECIMAL(12,2);
ALTER TABLE employees DROP COLUMN email;
ALTER TABLE employees ADD CONSTRAINT uq_email UNIQUE (email);

CREATE INDEX idx_name ON employees(emp_name);

DROP TABLE employees;        -- removes the TABLE
TRUNCATE TABLE employees;    -- empties the table, keeps the structure
```

SECTION 10 — DML — INSERT, UPDATE, DELETE

```
INSERT INTO employees (emp_id, emp_name, dept_id, salary)
VALUES (1, 'Asha', 1, 90000),
       (2, 'Ravi', 1, 70000);

UPDATE employees
SET    salary = salary * 1.10
WHERE  dept_id = 2;

DELETE FROM employees
WHERE  emp_name = 'Arjun';
```

Safety habit: run the same WHERE as a SELECT first, so you see exactly which rows you are about to change.

SECTION 11 — TRANSACTIONS

```
START TRANSACTION;
  UPDATE accounts SET balance = balance - 5000 WHERE id = 1;
  SAVEPOINT sp1;
  UPDATE accounts SET balance = balance + 5000 WHERE id = 2;
ROLLBACK TO sp1;     -- undo just the second update
COMMIT;              -- make the rest permanent   ( or: ROLLBACK; to undo all )
```

ACID: **A**tomicity (all or nothing), **C**onsistency, **I**solation, **D**urability.

Isolation levels, weakest to strongest: READ UNCOMMITTED -> READ COMMITTED -> REPEATABLE READ -> SERIALIZABLE.

SECTION 12 — THE DIFFERENCES (the direct "what's the difference" questions)

Pair	The difference
WHERE vs HAVING	WHERE filters ROWS before grouping and cannot use aggregates; HAVING filters GROUPS after grouping and CAN use aggregates
GROUP BY vs ORDER BY	GROUP BY collapses rows into groups; ORDER BY only sorts
DELETE vs TRUNCATE vs DROP	DELETE removes rows (rollback-able); TRUNCATE empties the whole table; DROP removes the table itself
UNION vs UNION ALL	UNION removes duplicates; UNION ALL keeps everything
IN vs EXISTS	membership in a list vs existence of a row
COUNT(*) vs COUNT(col)	rows (NULLs included) vs non-NULL values only
RANK vs DENSE_RANK	skips after a tie (1,2,2,4) vs does not skip (1,2,2,3)
PRIMARY vs UNIQUE	never NULL, one per table vs one NULL allowed, many per table
Clustered vs Non-clustered	the table stored in index order vs a separate index with row pointers

SECTION 13 — THE QUERIES THEY ASK FOR (visible syntax)

```
-- 2nd highest salary (subquery)
SELECT MAX(salary) FROM employees
WHERE  salary < (SELECT MAX(salary) FROM employees);

-- 2nd highest salary (LIMIT way)
SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 1;

-- Nth highest salary
SELECT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk FROM employees
) t
WHERE rnk = 2;

-- duplicate rows
SELECT emp_name, COUNT(*) FROM employees GROUP BY emp_name HAVING COUNT(*) > 1;

-- employees earning more than their manager
SELECT w.emp_name FROM employees w
JOIN   employees m ON w.manager_id = m.emp_id
WHERE  w.salary > m.salary;

-- departments with NO employees
SELECT d.dept_name FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
WHERE  e.emp_id IS NULL;

-- above the company average
SELECT emp_name FROM employees
WHERE  salary > (SELECT AVG(salary) FROM employees);

-- running total
SELECT hire_date, SUM(salary) OVER (ORDER BY hire_date) AS running_total
FROM   employees ORDER BY hire_date;

-- top salary per department
SELECT dept_id, MAX(salary) FROM employees GROUP BY dept_id;

-- count per department, INCLUDING zero
SELECT d.dept_name, COUNT(e.emp_id)
FROM   departments d LEFT JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name;
```

SECTION 14 — HANDY FUNCTIONS

```
-- STRING
CONCAT(first, last)      LENGTH(s)        UPPER(s)      LOWER(s)
SUBSTRING(s, 1, 3)       TRIM(s)          REPLACE(s, 'a', 'b')

-- DATE
CURDATE()                NOW()            YEAR(d)       MONTH(d)
DATEDIFF(d1, d2)         DATE_ADD(d, INTERVAL 30 DAY)   DATE_FORMAT(d, '%Y-%m')

-- NUMERIC
ROUND(x, 2)              CEIL(x)          FLOOR(x)      ABS(x)      MOD(a, b)

-- NULL and CONDITIONAL
COALESCE(col, 0)         -- replace NULL with 0
CASE WHEN salary >= 80000 THEN 'High'
     WHEN salary >= 60000 THEN 'Medium'
     ELSE 'Low' END      -- build a conditional column
```

SECTION 15 — POWER COMBOS

```
-- filter rows, group, filter groups, sort
SELECT   dept_id, AVG(salary) AS avg_sal
FROM     employees
GROUP BY dept_id
HAVING   AVG(salary) > 60000
ORDER BY avg_sal DESC;

-- rank within each group, then filter outside
SELECT * FROM (
  SELECT *, DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
  FROM employees
) t
WHERE rnk <= 2;

-- keep the empty groups (LEFT JOIN + COUNT of the column)
SELECT d.dept_name, COUNT(e.emp_id)
FROM   departments d LEFT JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name;

-- find what is missing (anti-join)
SELECT d.dept_name FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
WHERE  e.emp_id IS NULL;

-- label rows with CASE
SELECT emp_name,
       CASE WHEN salary >= 80000 THEN 'High' ELSE 'Normal' END AS band
FROM   employees;
```

MEMORY HOOK: *"Filter rows, group, filter groups, sort — and every difference question is just two of those steps compared."*